

Introduction to OOP

Class & Objects

- Class is blueprint
- Contains data or property or attributes & functions or behaviours.
- Object is instance of class

Syntax `Obj-name = classname()`

eg: `L = list()`

- Class are always written in pascal case eg: HelloWorld
- Two types of classes in Python
 - ① Built in classes
 - ② User defined classes.

Methods vs functions

- The functions created inside class are called methods
- The function that exists independently outside of class are called functions

eg: `L = [1, 2, 3]`

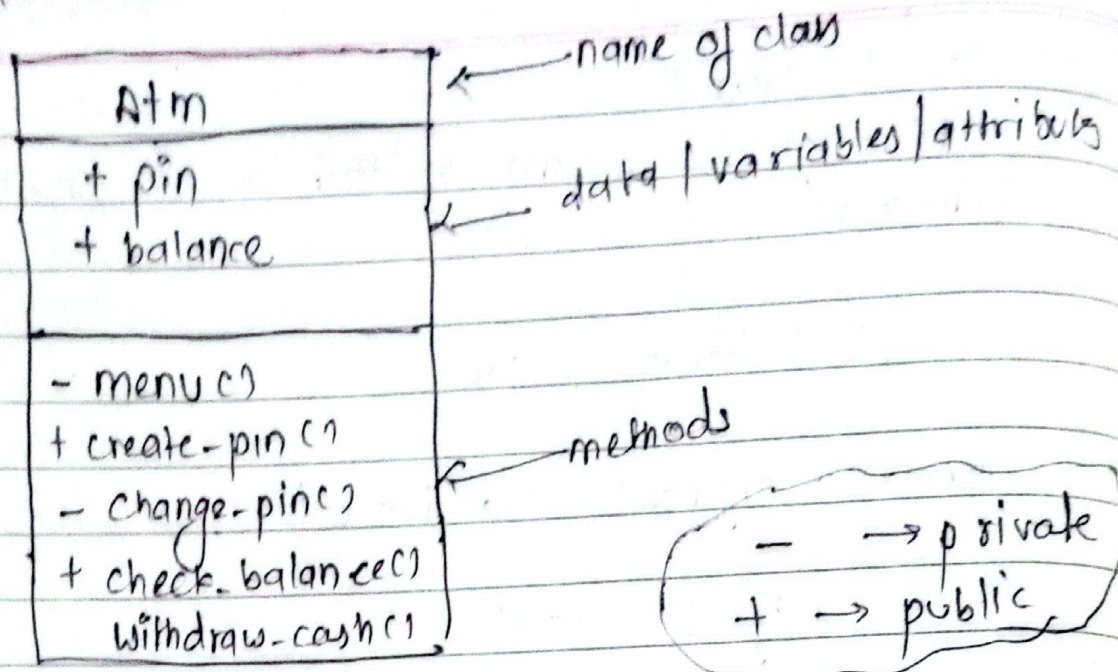
`len(L)`

→ function → it is outside class
list

`L.append()`

→ method → it is inside class
list

Dunder methods class diagram



Magic functions (dunders) (dunder methods)

→ Special methods, have superpower

-- name of method --

eg: --init--

seems like

constructor
superpower: it is called itself
whenever object of class
is created you don't have
to call it separately

① using these methods you can create your own datatypes

(1) Constructor

- ↳ method, which is triggered itself.
- used to write configuration related code.
- The things that ~~you~~ are not controlled by users are written inside constructors
- e.g. database connection

(2) (*) self

→ members of the class can only be accessed by the object of that class

→ Golden rule of oop

one method inside class cannot call another method inside same class

→ Self is object

→ For communication betⁿ two methods inside class
(methods cannot communicate directly)

↳ first a method uses a object of that class to access another method. for this purpose self is used.

→ self is current object.

(this is used in c++)

② Encapsulation & static key word

class Person:

```
def __init__(self, name, country):  
    self.name = name  
    self.country = country
```

```
def greet(self):  
    if self.country == 'Nepal':  
        print('Namaste', self.name)  
    else:  
        print('Hello', self.name)
```

(*) to access attribute

```
p = Person('Lokesh', 'Nepal')
```

```
p.name  
p.country
```

p is not a
obj. a
contains
reference of
address of
object

to access attributes

(*) to access methods

```
p.greet()
```

to access methods

(*) to access non-existent attribute
p.gender → error

(*) attribute creation from outside class

```
p.gender = 'male'
```

```
p.gender
```

You can create
attribut outside
the class

Reference variable: (Object)

variable

In

$p = \text{person}('Lokesh', 'Nepal')$

→ p is not a object. It is reference variable that is pointing to the reference or address of the object $\text{person}('Lokesh', 'Nepal')$ in memory.

- reference variables hold the objects
- We can create objects without reference variable as well
- An object can have multiple reference variables
- Assigning a new reference variable to an existing object does not create a new object.

Pass by reference

Encapsulation

of variable

* Instance variables (value depends on object)
↳ variables that have different value for different objects.

private variables

→ we can make private variable as follow
-- balance

→ It is stored in memory as

-- classname -- ~~balance~~ variable name

so if we access ~~or~~ try to access variable as

-- variable name → It create new variable
but the private variable is not accessed.

→ Nothing is truly private in python because python is programming language for adults. [stackoverflow]

→ ~~object~~ we can access private variable from methods of class

→ getter & setter method

shows private value outside class

change private's value from outside class

→ This is concept of encapsulation used when

you have to private your data as well as you have to show or make it edit outside class

Static variable vs Instance variable

Static variable is object's variable

Instance variable is class's variable

→ static variable is same for each variable

→ instance variable is different for each variable.

self.variable-name

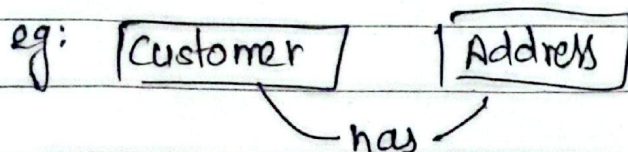
instance

classname.variable name

static

Class Relationships

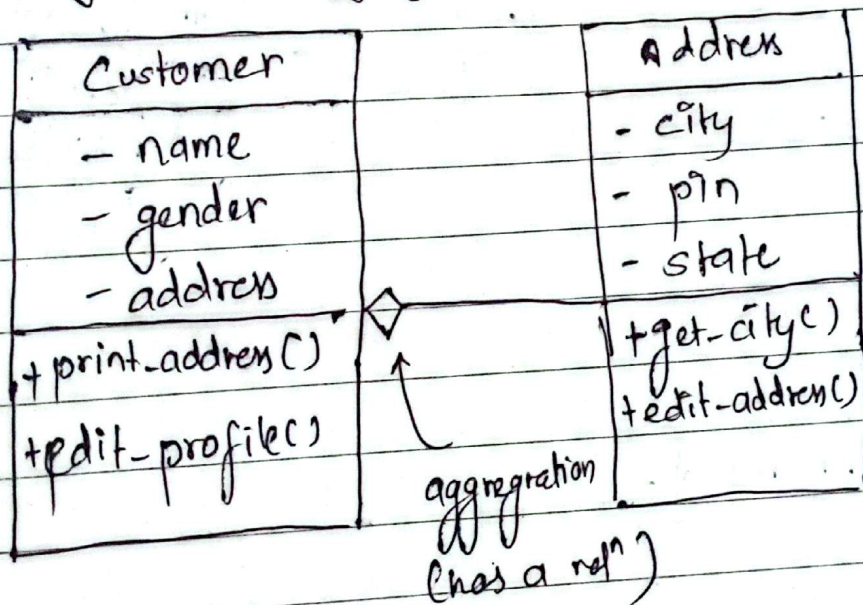
① Aggregation: has a relⁿ (one class owns another class)



so customer class owns address class

→ cannot access private variable of the another class in aggregation.

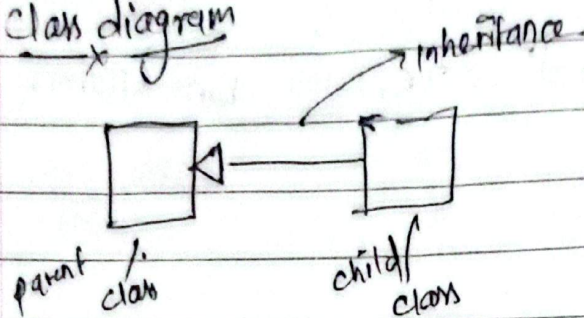
class diagram of aggregation



Inheritance

- child class can access ~~every~~ attribute & methods of parent class
- It ~~seems~~ is effective because we can reuse code through inheritance

class diagram



class User:
body

parent

class student(User):
body

child

syntax
for inheritance
(x -> y)

can access non
private attribute
& methods of
parent class

what gets inherited.

(1) Constructor

→ If child class does not have its own constructor then the constructor of parent class is called

→ If child class has its own constructor then ~~the~~ while creating obj of child class its own constructor is called, the parent class constructor is not called.

- ② non-private methods & variables
→ child class's object cannot access private members of parent class

method overriding

- If both parent & child class has same method (same method name) then if child class's object call that function, the method of child class will be executed.

```
class parent:
    def show():

class child:
    def show():

c = child()
c.show()
```

this is called.

Super keyword

- Using super keyword you can call the parents ~~attributes~~ methods in child class.

```
class parent:
    def show():
        body

class child:
    def show():
        body
        super().show()

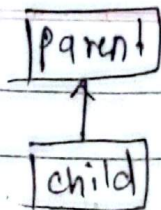
c = child()
c.show()
```

calls

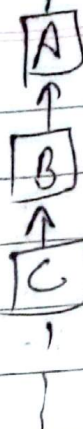
- super() keyword doesnot work outside the child's class
→ Super() keyword cannot access the attributes of the parent class. Only methods can be accessed.

Types of Inheritance

① Single



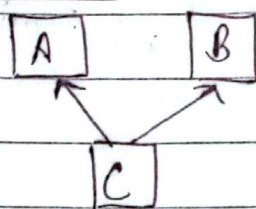
② Multilevel



→ chain could be long from both sides

→ child inherit property of every parent above it (C inherit A & B)

③ Multiple



→ multiple parents

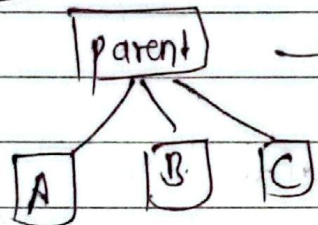
→ one children

→ If A & B has same method & obj of C calls the method. the method of the first class will be executed according to method resolution order

~~If class B (B, A)~~

If class B (B, A) priority to B

④ Hierarchical

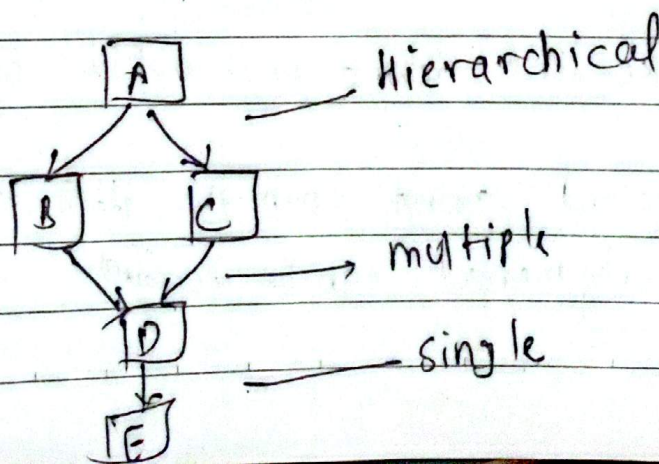


→ one parent

→ ~~single child~~
multiple children

⑤ Hybrid

combⁿ of different types of Inheritance.



Hierarchical

multiple

single

Polymorphism:

→ same things behave differently according to the situation.

→ having multiple faces

① method overriding

→ child class's method override the parent class method if the method is same in both class when called by object of child class.

② method overloading (not used in python)

→ The name of the method are same but they differ by the number of arguments passed.

eg: class shape:

```
def area (self, radius):  
    return 3.1415 * radius ** 2
```

```
def area (self, l, b):  
    return l * b
```

→ More cleaner code / readability

→ This concept is not in python

(reason behind overloading)

③ Operator overloading
→ Same operator behaves differently if you vary the input.

eg:

'Hello' + 'world' = Helloworld

4 + 5 = 9

[1, 2, 3] + [4, 5] = [1, 2, 3, 4, 5]

Abstraction

→ hiding complex details, shows surface level

* Abstract class

contains atleast one abstract method (no code)
(it contains no implementation)

• module

from abc import ABC, abstractmethod.

→ abstract method can be made using

(abstract method) decorator.

* To be abstract class

→ first class should inherit ABC class

→ class should contain atleast one abstract method.

→ You cannot make object of abstract class.

→ So to inherit another class from this abstract class

It must contain abstract method made inside abstract class

eg:

```
from abc import ABC, abstractmethod  
class BankApp(ABC):
```

```
    def database(self):  
        print('connect to database')
```

```
    @abstractmethod  
    def security(self):  
        pass
```

abstract
class

abstract method

```
class MobileApp(BankApp):
```

```
    def mobile_login(self):  
        print('login into mobile')
```

```
mob = MobileApp()
```

→ It will throw an error as it does not have security() method

→ you have to implement security() method inside mobileApp class & write some code to inherit from parent class

→ parent class can make do something to child class if they want to inherit parent class.

Concrete ~~class~~ method

→ Normal method we are making till now.
that contains body

abstract method

→ That is inside abstract class, donot have body.

→ The child class who want to inherit that abstract class must implement that abstract method.